

一、云服务器端流程

第 1 步：进入云端环境

在云服务器上进入你已经装好 vLLM/Qwen 的环境：

```
conda activate model_qwen_310
```

第 2 步：设置 CUDA/链接环境

每次开新终端，建议先执行：

```
export CUDA_HOME="$CONDA_PREFIX"
export PATH="$CUDA_HOME/bin:$PATH"
export LD_LIBRARY_PATH="/usr/lib/x86_64-linux-gnu:$CUDA_HOME/lib:$CUDA_HOME/lib64:$LD_LIBRARY_PATH"
export LIBRARY_PATH="/usr/lib/x86_64-linux-gnu:$CUDA_HOME/lib:$CUDA_HOME/lib64:$LIBRARY_PATH"
export LDFlags="-L/usr/lib/x86_64-linux-gnu $LDFlags"
```

如果之前已经给 conda 环境里补过 libcuda.so 软链接，也保留。

第 3 步：清理 FlashInfer 缓存

当你改过环境或修过链接问题时，先清一次：

```
rm -rf ~/.cache/flashinfer
```

第 4 步：启动 vLLM 服务

你现在这套“单张图片正反面分类”的推荐启动命令是：

```
vllm serve /home/roo/.cache/modelscope/hub/models/Qwen/Qwen3.5-35B-A3B \
--host 127.0.0.1 \
--port 8000 \
--served-model-name Qwen/Qwen3.5-35B-A3B \
--reasoning-parser qwen3 \
--default-chat-template-kwarg '{"enable_thinking": false}' \
--max-model-len 1024 \
--gpu-memory-utilization 0.6 \
--max-num-seqs 2 \
--max-num-batched-tokens 128 \
--limit-mm-per-prompt '{"image": 1}' \
```

--enforce-eager

```
vllm serve /home/roo/.cache/modelscope/hub/models/Qwen/Qwen3.5-35B-A3B \
--host 127.0.0.1 \
--port 8000 \
--served-model-name Qwen/Qwen3.5-35B-A3B \
--reasoning-parser qwen3 \
--default-chat-template-kwarg '{"enable_thinking": false}' \
--max-model-len 1024 \
--gpu-memory-utilization 0.72 \
--max-num-seqs 1 \
--max-num-batched-tokens 256 \
--limit-mm-per-prompt '{"image": 1}' \
--enforce-eager
```

这条命令的含义你前面已经看过，核心就是：

- 只服务单图请求
- 降低显存占用
- 降低并发
- 关闭 thinking
- 优先稳定

第 5 步：确认服务启动成功

服务终端里看到类似：

- Started server process
- Application startup complete

再开云端另一个终端测试：

```
curl http://127.0.0.1:8000/v1/models
```

如果能返回：

```
"Qwen/Qwen3.5-35B-A3B"
```

说明云端服务已起来。

二、本机端流程

第 1 步：建立 SSH 隧道

在本机开一个终端，执行：

```
ssh -L 8000:127.0.0.1:8000 roo@192.168.5.37
```

这条命令执行后，这个终端不要关。

作用是：

- 本机访问 127.0.0.1:8000
- 实际转发到云服务器 127.0.0.1:8000

第 2 步：验证本机到云端 Qwen 已通

在本机另一个终端执行：

```
curl http://127.0.0.1:8000/v1/models
```

如果返回模型列表，说明隧道正常。

三、本机程序准备

你的主程序现在已经支持：

- `--remote_api_base`
- `--remote_model_name`
- `--remote_api_key`

并且默认会从 `/v1/models` 自动解析模型名。

第 1 步：进入本机运行环境

进入你能正常跑 FoundationPose / Orbbec / 分割 / 机器人控制的环境，比如：

```
conda activate fp
```

第 2 步：确认工程依赖齐全

这份程序依赖这些模块：

- `estimator`
- `datareader`
- `Cameras.OrbbecCamera`
- `ImageCoarseSegmentor_pointcloud`
- `Robots.robot_control`

以及默认资源路径：

- `demo_data_pian/my_data0/mesh/pian_hole_m.obj`
- `templates/back`

这些都在程序里写死了。

第 3 步：确认相机和机器人状态

你当前这版代码里：

- 机器人初始化已开启
- 相机初始化必须成功

也就是：

- 机器人没连，可能打印警告，但不一定退出
- 相机没连，程序会直接失败

如果你现在只是测试视觉，建议把机器人初始化暂时关掉，改回：

```
print(">>> 跳过机器人初始化（仅测试视觉）...")  
self.robot = None
```

这样更稳。

四、本机程序运行

启动命令

在本机项目目录执行：

```
python pick_place_qwen_vllm_remote.py \  
  --remote_api_base http://127.0.0.1:8000/v1 \  
  --remote_model_name auto \  
  --remote_api_key EMPTY
```

如果文件名不是这个，就换成你的实际文件名。